

openGauss中存储过程、触发器与游标应用

难度（最高三星）：★★★ 建议学时（2-4学时）：4学时

实验说明

学习地图

本项目实验案例集是实现一个学生成绩管理系统，共分为9个实验任务，按照难度等级分为初级、中级、高级，如图1所示。



图1 学生成绩管理系统实验

本次实验主要完成其中的任务4，主要实现学生管理系统中存储过程、触发器以及游标的使用。

实验环境准备

(1) openGauss的安装与部署：该部分请参照实验任务1【[openGauss表空间、模](#)

[式、权限管理](#)】进行安装与部署。

(2) 数据库创建：请参照实验任务1【[openGauss表空间、模式、权限管理](#)】，或[直接下载相关SQL语句](#)进行数据库表空间创建、数据库模式创建、用户权限管理。

(3) 学生成绩管理系统的数据表以及相关约束：请参照实验任务2【[openGauss数据表创建与数据表约束](#)】，或[直接下载相关SQL语句](#)并进行运行生成相关的表。

(4) 对学生成绩管理系统中的表数据进行插入：请参照实验任务3【[openGauss数据库SQL语法](#)】，或[直接下载相关SQL语句](#)并进行运行生成相关的表。

实验任务

任务描述

本次实验共计四个子任务，主要是完成对学生成绩管理系统的数据表使用存储过程、触发器、游标技术。整个实验会有详细的需求说明和实施步骤和关键代码。

学习目标

完成本任务的学习后，你应当能：

1. 学会使用存储过程完成数据查询、插入；
2. 学会创建触发器完成数据更新；
3. 学会使用游标实现数据库查询操作。

任务准备

前置知识

本实验案例，需提前学习openGauss的基本概念、作用及基础操作。可通过如下途径进行学习：

- 数字教材：《[数据库原理与应用：基于openGauss](#)》
- 技术文档：[openGauss白皮书](#)

任务实施

任务1：使用存储过程查询指定学期开设的课程数和总课时。

本实验的具体操作可参见视频。

训练要点：使用输入、输出参数的存储过程完成数据查询，获得相关数据。

需求说明如下：

- (1) 查询获得指定学期开设的课程数和总课时。
- (2) 如果学期名称为空，则显示“学期名称不能为空”，并返回。

实现思路如下：

- (1) 输入参数是学期名称。
- (2) 输出参数是课程数和总课时数。
- (3) 查询获得指定学期所开设的课程、课时。
- (4) 查询该学期的总课程数、总课时数。

关键代码如下：

(1) 创建存储过程

```
CREATE OR REPLACE PROCEDURE get_term_course_info(  
    IN term_name VARCHAR(50),  
    OUT course_count INTEGER,  
    OUT total_hours NUMERIC  
)  
AS  
BEGIN  
    -- 检查是否输入了学期名称  
    IF term_name IS NULL THEN  
        RAISE NOTICE '学期名称不能为空';  
        course_count := 0;  
        total_hours := 0;  
        RETURN;  
    END IF;  
    -- 查询指定学期的课程数和总课时  
    SELECT  
        COUNT(*),  
        COALESCE(SUM(s.Class_Hour), 0)  
    INTO course_count, total_hours  
    FROM Subject s  
    INNER JOIN Grade g ON s.Grade_Id = g.Grade_Id  
    WHERE g.Grade_Name = term_name;  
    RAISE NOTICE '对于学期 "%", 共有 % 节课程, 总学时为 % 小时。', term_name,  
course_count, total_hours;  
END;  
/
```

(2) 调用存储过程 (使用 PERFORM 调用存储过程的原因是为了在存储过程没有返回值时避免出现"查询没有目标接收结果数据"的错误)。

```
DECLARE  
    v_course_count INTEGER;  
    v_total_hours NUMERIC;
```

```
BEGIN
    PERFORM get_term_course_info(NULL, v_course_count, v_total_hours);
END;
/
```

查询结果如下：

学期名称不能为空。

```
DECLARE
    v_course_count INTEGER ;
    v_total_hours NUMERIC;
BEGIN
    PERFORM get_term_course_info('大一年级', v_course_count, v_total_hours);
END;
/
```

查询结果如下：

对于学期 ‘大一年级’ ， 共有4节课程，总学时为156小时。

任务2：使用存储过程插入新课程记录

本实验的具体操作可参见视频。

训练要点：使用输入、输出参数的存储过程完成数据查询和插入。

需求说明如下：

- (1) 增加指定学期开设的课程记录。
- (2) 如果学期名称或课程名称为空，则终止存储过程执行返回。
- (3) 如果课程所属学期不存在，要添加学期记录。
- (4) 返回增加的课程编号对应的学期编号。
- (5) 获得操作的结果，并根据不同的执行结果显示相应的提示信息。

实现思路如下：

- (1) 将要增加的课程数据作为输入参数。
- (2) 新增的课程编号作为输出参数。
- (3) 增加记录的执行结果作为返回值。
- (4) 先检查和增加学期记录，后增加课程记录。

关键代码如下：

- (1) 创建存储过程

```
CREATE OR REPLACE PROCEDURE add_course_for_term(  
    IN p_term_name VARCHAR(50),  
    IN p_course_name VARCHAR(100),  
    IN p_class_hour NUMERIC,  
    OUT o_course_id INTEGER,  
    OUT o_term_id INTEGER,  
    OUT o_result BOOLEAN  
)  
AS  
DECLARE  
    v_term_id INTEGER;  
BEGIN  
    -- 检查学期名称和课程名称是否为空  
    IF p_term_name IS NULL OR p_course_name IS NULL THEN  
        RAISE NOTICE '学期名称和课程名称不能为空。';  
        o_result := FALSE;  
        RETURN;  
    END IF;  
    -- 查找学期记录  
    SELECT Grade_Id INTO v_term_id  
    FROM Grade  
    WHERE LOWER(Grade_Name) = LOWER(p_term_name);  
    -- 如果学期记录不存在,则插入新记录
```

```
IF v_term_id IS NULL THEN
    INSERT INTO Grade (Grade_Name)
    VALUES (p_term_name)
    RETURNING Grade_Id INTO v_term_id;
    RAISE NOTICE '插入新学期: %', p_term_name;
END IF;
-- 插入课程记录
INSERT INTO Subject (Grade_Id, Subject_Name, Class_Hour)
VALUES (v_term_id, p_course_name, p_class_hour)
RETURNING subject_no INTO o_course_id;
o_term_id := v_term_id;
o_result := TRUE;
RAISE NOTICE '添加新课程: %, 学期: %', p_course_name, p_term_name;
END;
/
```

(2) 调用存储过程

```
DECLARE
    v_course_id INTEGER;
    v_term_id INTEGER;
    v_result BOOLEAN;
BEGIN
    PERFORM add_course_for_term('大一年级', '大学英语', 64, v_course_id, v_term_id,
v_result);
    PERFORM add_course_for_term(NULL, '大学物理', 72, v_course_id, v_term_id, v_result);
END;
```

查询结果如下：

添加新课程：大学英语，学期：大一年级
学期名称和课程名称不能为空。

任务3：使用触发器记录成绩更新情况

本实验的具体操作可参见视频。

训练要点：创建并使用存储过程和触发器。

需求说明如下：

- (1) 设计一个触发器和存储过程，用于记录成绩表 Result 中成绩的更新情况。
- (2) 记录学号、课程编号、原始成绩、新成绩以及更新时间等信息。

实现思路与关键代码如下：

- (1) 创建成绩更新记录表（日志表）ScoreUpdateLog。

```
CREATE TABLE ScoreUpdateLog (  
    Id SERIAL PRIMARY KEY,  
    Student_No INT NOT NULL,  
    Subject_No INT NOT NULL,  
    OldScore INT NOT NULL,  
    NewScore INT NOT NULL,  
    UpdateTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

- (2) 创建触发器函数 log_score_update()。

```
CREATE OR REPLACE FUNCTION log_score_update()  
RETURNS TRIGGER AS $$  
BEGIN  
    -- 记录更新前的成绩信息  
    INSERT INTO ScoreUpdateLog (Student_No, Subject_No, OldScore, NewScore)  
    VALUES (OLD.Student_No, OLD.Subject_No, OLD.Student_Result, NEW.Student_Result);  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

- (3) 创建触发器并将其附加在Result表上。

```
CREATE TRIGGER tr_log_score_update  
AFTER UPDATE ON Result  
FOR EACH ROW  
EXECUTE PROCEDURE log_score_update();
```


(4) 测试该触发器，插入新的数据不会触发该触发器，因为没有旧的成绩。

```
INSERT INTO Result (Student_No, Subject_No, Student_Result, Exam_Date)
VALUES (10001, 6, 90, '2024-02-17 00:00:00');
```

验证ScoreUpdateLog 表是否为空，若为0则测试通过。

```
SELECT COUNT(*) FROM ScoreUpdateLog;
```

更新原有的成绩，触发该触发器。

```
UPDATE Result
SET Student_Result = 80
WHERE Student_No = 10000 AND Subject_No = 3 AND Exam_Date='2024-02-17 00:00:00';
```

此时再查看ScoreUpdateLog表，如图2所示。

```
SELECT * FROM ScoreUpdateLog;
```

	id	student_no	subject_no	oldscore	newscore	updateime
1	1	10000	3	60	80	2024-08-07 21:39:19

图2 ScoreUpdateLog表

任务4：使用存储过程和游标查看某个学生的成绩情况

本实验的具体操作可参见视频。

训练要点：使用存储过程和游标实现数据库查询操作。

需求说明如下：

- (1) 输入学生的学号，查看该学生的所有课程和对应的成绩。
- (2) 查看该学生的最高分和是否存在不及格科目。

实现思路及关键代码如下：

- (1) 创建存储过程和游标。

```
CREATE OR REPLACE PROCEDURE get_student_result(
    p_student_no INT
)
```

```
AS
    v_student_name VARCHAR(50);
    v_max_score INT := 0;
    v_max_subject VARCHAR(50);
    v_has_fail BOOLEAN := FALSE;
    CURSOR c_student_result IS
        SELECT s.Subject_Name, r.Student_Result
        FROM Result r
        JOIN Subject s ON r.Subject_No = s.Subject_No
        WHERE r.Student_No = p_student_no;
    rec RECORD;
BEGIN
    -- 查询学生姓名
    SELECT student_name INTO v_student_name
    FROM student
    WHERE student_no = p_student_no;
    -- 输出学生姓名
    RAISE NOTICE '学生姓名: %', v_student_name;
    -- 打开游标
    OPEN c_student_result;
    FETCH c_student_result INTO rec;
    WHILE c_student_result%FOUND LOOP
        RAISE NOTICE '科目: %, 成绩: %', rec.Subject_Name, rec.Student_Result;
        -- 记录最高分及对应科目
        IF rec.Student_Result > v_max_score THEN
            v_max_score := rec.Student_Result;
            v_max_subject := rec.Subject_Name;
        END IF;
        -- 检查是否有不及格科目
        IF rec.Student_Result < 60 THEN
            v_has_fail := TRUE;
        END IF;
        FETCH c_student_result INTO rec;
    END LOOP;
```

```
-- 关闭游标
CLOSE c_student_result;

-- 输出最高分及对应科目
RAISE NOTICE '最高分: %, 对应科目: %', v_max_score, v_max_subject;

-- 如果有不及格科目,给出提示
IF v_has_fail THEN
    RAISE NOTICE '该学生存在不及格科目';
END IF;

END;

/
```

(2) 调用该存储过程

```
CALL get_student_result(10002);
```

查询结果如下:

```
学生姓名: Charlie
科目: Java编程基础, 成绩: 73
科目: C语言基础, 成绩: 83
最高分: 83, 对应科目: C语言基础
```

```
CALL get_student_result(30021);
```

查询结果如下:

```
学生姓名: Isabella
科目: 数据库高级特性, 成绩: 23
科目: Java数据库实战, 成绩: 74
最高分: 74, 对应科目: Java数据库实战
该学生存在不及格科目
```

随堂测试

1. 有关存储过程的参数默认值, 下面说法正确的是()

A. 输入参数必须有默认值

- B. 带默认值的输入参数，方便用户调用
- C. 带默认值的输入参数，用户不能再传入参数，只能采用默认值
- D. 输出参数也可以带默认值

2. 下面有关存储过程的说法，()是错误的

- A. 它可作为作为一个独立的数据库对象并作为一个单元供用户在应用程序中调用
- B. 存储过程可以传入和返回(输出)参数值
- C. 存储过程必须带参数，要么是输入参数，要么是输出参数
- D. 存储过程提高了执行效率

3. 在openGauss触发器中，触发事件不包括()

- A. INSERT B. TRUNCATE C. UPDATE D. CHECK

4. 用于判断游标是否成功提取到数据的属性是()

- A. %ROWCOUNT B. ROWCOUNT C. %FOUND D. ISFOUND

5. 修改触发器的语句是()

- A. CREATE TRIGGER
- B. CREATE OR REPLACE TRIGGER
- C. ALTER TRIGGER
- D. MODIFY TRIGGER